

Order Supervisor

Overview

Build a POC for a long-running AI supervisor that oversees a single order from creation until completion.

The system should start one long-running Temporal workflow per order. As events happen for that order, they should be delivered into the workflow. The AI should decide when to act immediately, when to go back to sleep, and when to wake up later to review the situation again.

This assignment is intended to be completed in 1-2 days. We care most about system design, Temporal usage, agent orchestration, and the ability to build a small but working end-to-end product.

Problem Statement

Assume an order has been placed.

We want an AI supervisor that:

- watches the lifecycle of that order,
- receives updates over time,
- maintains memory and history,
- decides when intervention is needed,
- executes actions through tools,
- sleeps when no action is needed,
- wakes up again on schedule or when important events happen,
- produces final learnings and feedback when the workflow ends.

Core Idea

Create one workflow run per order.

The workflow should support 3 triggers for agent inference:

- workflow start,
- incoming event or signal,
- scheduled wake-up.

The AI should not run continuously in a tight loop. It should be able to:

- reason,
- take actions,
- update its memory,
- set wake-up instructions,
- go to sleep,
- wake up later when needed.

Required Stack

- Frontend: Next.js with App Router
- UI: Tailwind CSS
- Backend: Python with FastAPI
- Orchestration: Temporal Python SDK (`temporalio`)
- Persistence: PostgreSQL, Supabase is fine
- LLM/tool orchestration: your choice

What You Need to Build

1. Long-Running Order Workflow

When an order is created:

- start a Temporal workflow for that order,
- initialize it with order context and a base instruction,
- keep the workflow alive until the order reaches a terminal state.

Events related to that order should be sent into the workflow as signals.

2. Agent Runtime

The agent should:

- run when the workflow starts,
- run when an important signal arrives,
- run when its scheduled wake-up time is reached,
- decide whether to act,
- call tools when needed,
- update memory and timeline,
- decide when to sleep again.

3. Event-Driven Wake/Sleep Behavior

The system should support:

- sleeping until a scheduled time,
- waking on incoming signals,
- deciding whether an event is important enough to wake the main agent.

A simple implementation is acceptable.

For example:

- every incoming event is first checked by a lightweight classifier or policy,
- the classifier decides whether to wake the main agent now,
- otherwise the workflow stays asleep until the next scheduled wake-up.

4. Memory and Timeline

Each run should maintain:

- a timeline of important events and actions,
- a compact memory summary,
- current workflow status,
- current sleep state or next wake-up time.

The system should also support simple context compaction, for example by:

- summarizing older history,
- storing only important events in memory,
- maintaining a rolling summary.

A sophisticated memory architecture is not required.

5. Tool Calls

The agent should be able to execute tools.

Implement at least 4 tools such as:

- send customer message,
- create internal note,
- escalate issue,
- mark order for review,
- schedule next wake-up,
- close workflow.

The tools can be mocked or simulated. Real external integrations are not required.

6. Event Generator

Build a simple event generator or simulator that can send events into a workflow run.

Possible events:

- `order_created`
- `payment_confirmed`
- `payment_failed`
- `shipment_created`
- `shipment_delayed`
- `delivered`

- `refund_requested`
- `customer_message_received`
- `no_update_for_n_hours`

The event generator can be:

- a small UI panel,
- a backend endpoint,
- a script,
- or all of the above.

7. UI

Build a small UI where a user can:

- configure a supervisor template,
- start a run for an order,
- view active and completed runs,
- inspect timeline and action history,
- inspect memory summary,
- inject events,
- add extra instructions to a specific run,
- interrupt, pause, resume, or terminate a run.

The UI does not need to look polished. It should be functional and easy to understand.

Supervisor Configuration

The user should be able to define a supervisor setup with:

- name,
- base instruction,
- available tools,

- optional default wake-up behavior,
- optional model choice or config,
- optional guidance for how aggressively the agent should wake up.

Hardcoded templates are acceptable.

Additional Instructions Per Run

The system should allow users to add run-specific instructions after a workflow has already started.

Example:

- "For this order, prioritize speed over cost."
- "If shipment is delayed, escalate immediately."
- "Do not contact the customer without human review."

These instructions should become part of the run context.

End-of-Run Output

When the workflow finishes, the system should generate:

- final summary,
- important actions taken,
- key learnings,
- feedback or recommendations.

This can be produced by the main agent as a final step.

Good-to-Have

These are not mandatory, but are strong additions:

- agent-generated wake-up guidance for the classifier,
- unknown-event escalation behavior,
- `continue_as_new` for very long histories,

- better memory compaction strategy,
- multiple supervisor templates,
- richer run analytics.

Suggested System Design

A clean implementation could have:

- FastAPI backend for workflow/run management,
- Temporal workflow per order,
- signal handlers for incoming events,
- one main agent runtime,
- one lightweight classifier or wake-up policy,
- persistence for runs, timeline, memory, and final output,
- Next.js UI for configuration and monitoring.

Suggested APIs

Your exact API design is up to you, but a clean version could include:

- `POST /api/supervisors`
- `GET /api/supervisors/{id}`
- `POST /api/runs`
- `GET /api/runs`
- `GET /api/runs/{run_id}`
- `POST /api/runs/{run_id}/events`
- `POST /api/runs/{run_id}/instructions`
- `POST /api/runs/{run_id}/interrupt`
- `POST /api/runs/{run_id}/resume`
- `POST /api/runs/{run_id}/terminate`

Acceptance Criteria

A submission is successful if:

- one Temporal workflow is started per order,
- order events can be sent into the workflow as signals,
- the agent can wake on start, signal, and scheduled wake-up,
- the agent can sleep and wake later,
- the agent can call tools,
- the run stores timeline and compact memory,
- the UI can show run history and current state,
- the UI can inject events and additional instructions,
- the workflow produces a final summary with learnings and feedback.

Scope Boundaries

You do not need to build:

- real commerce integrations,
- real messaging integrations,
- authentication,
- multi-tenant production hardening,
- advanced retrieval systems,
- multiple cooperating agents,
- polished design.

Keep the scope small and solid.

Deliverables

Please submit:

- source code,

- a README with setup instructions,
- a short architecture note,
- a walkthrough video showing the product working.

The walkthrough should show:

- creating a supervisor config,
- starting an order run,
- sending events into the workflow,
- the agent going to sleep and waking up,
- tool execution,
- adding extra instructions to a live run,
- interrupting or terminating a run,
- final summary, learnings, and feedback.

Evaluation Criteria

We will look at:

- architecture and clarity of design,
- Temporal usage,
- long-running workflow modeling,
- signal handling,
- quality of agent orchestration,
- memory/timeline design,
- frontend usability,
- code quality,
- documentation,
- overall product thinking.

Notes

A smaller implementation with a clean architecture and a reliable demo is better than a larger unfinished system.

Focus on building a believable POC, not a production platform.

Order Supervisor

Overview

Build a POC for a long-running AI supervisor that oversees a single order from creation until completion.

The system should start one long-running Temporal workflow per order. As events happen for that order, they should be delivered into the workflow as signals. The AI should decide when to act immediately, when to go back to sleep, and when to wake up later to review the situation again.

This assignment is intended to be completed in 1-2 days. We care most about system design, Temporal usage, agent orchestration, and the ability to build a small but working end-to-end product.

Problem Statement

Assume an order has been placed.

We want an AI supervisor that:

- watches the lifecycle of that order,
- receives updates over time,
- maintains memory and history,
- decides when intervention is needed,
- executes actions through tools,
- sleeps when no action is needed,
- wakes up again on schedule or when important events happen,
- produces final learnings and feedback when the workflow ends.

Core Idea

Create one workflow run per order.

The workflow should support 3 triggers for agent inference:

- workflow start,
- incoming event or signal,
- scheduled wake-up.

The AI should not run continuously in a tight loop. It should be able to:

- reason,
- take actions,
- update its memory,
- decide when to sleep,
- wake up later when needed.

Required Stack

- Frontend: Next.js with App Router
- UI: Tailwind CSS
- Backend: Python with FastAPI
- Orchestration: Temporal Python SDK (`temporalio`)
- Persistence: PostgreSQL, Supabase is fine
- LLM/tool orchestration: your choice

What You Need to Build

1. Long-Running Order Workflow

When an order is created:

- start a Temporal workflow for that order,
- initialize it with order context and a base instruction,

- keep the workflow alive until the order reaches a terminal state or is manually terminated.

Events related to that order should be sent into the workflow as signals.

2. Agent Runtime

The agent should:

- run when the workflow starts,
- run when an important signal arrives,
- run when its scheduled wake-up time is reached,
- decide whether to act,
- call actions when needed,
- update memory and timeline,
- decide when to sleep again.

3. Event-Driven Wake/Sleep Behavior

The system should support:

- sleeping until a scheduled time,
- waking on incoming signals,
- deciding whether an event is important enough to wake the main agent.

A simple implementation is acceptable.

For example:

- every incoming event is first checked by a lightweight classifier or policy,
- the classifier decides whether to wake the main agent now,
- otherwise the workflow stays asleep until the next scheduled wake-up.

It is good to have, but not mandatory, for the main agent to generate or refine wake-up guidance for future events.

4. Memory and Timeline

Each run should maintain:

- a timeline of important events and actions,
- a compact memory summary,
- current workflow status,
- current sleep state or next wake-up time.

The system should also support simple context compaction, for example by:

- summarizing older history,
- storing only important events in memory,
- maintaining a rolling summary.

A sophisticated memory architecture is not required.

Required Actions

The agent should be able to execute business actions.

Implement these 5 actions:

- `message_fulfillment_team`
- `message_payments_team`
- `message_logistics_team`
- `message_customer`
- `create_internal_note`

These actions do not need to send anything externally.

Instead:

- each action should create an activity record for the current workflow run,
- the activity should be stored in the database,
- the UI should display these activities in the run history.

A separate messages table is not required. It is enough to store all agent actions in a single activity log for the run.

Runtime Capabilities

In addition to the business actions above, the agent should also be able to:

- sleep for a duration or until a timestamp,
- update or refresh the compact memory summary,
- record its reasoning outcome in workflow state.

You may implement these as tools, workflow methods, or another clean abstraction.

Workflow Completion

The workflow should not end only because the AI decides to end it.

Instead, workflow completion should be tied to explicit lifecycle rules such as:

- a terminal order event arrives,
- the run is manually terminated from the UI,
- a configured max workflow age is reached,
- or another clearly defined completion rule.

The agent may recommend completion, but the workflow should ultimately be completed by workflow-owned rules.

Event Generator

Build a simple event generator or simulator that can send events into a workflow run.

Possible events:

- `order_created`
- `payment_confirmed`
- `payment_failed`
- `shipment_created`
- `shipment_delayed`

- `delivered`
- `refund_requested`
- `customer_message_received`
- `no_update_for_n_hours`

The event generator can be:

- a small UI panel,
- a backend endpoint,
- a script,
- or all of the above.

UI

Build a small UI where a user can:

- configure a supervisor template,
- start a run for an order,
- view active and completed runs,
- inspect timeline and activity history,
- inspect memory summary,
- inject events,
- add extra instructions to a specific run,
- interrupt, pause, resume, or terminate a run.

The UI does not need to look polished. It should be functional and easy to understand.

Supervisor Configuration

The user should be able to define a supervisor setup with:

- name,
- base instruction,

- available actions,
- optional default wake-up behavior,
- optional model choice or config,
- optional guidance for how aggressively the agent should wake up.

Hardcoded templates are acceptable.

Additional Instructions Per Run

The system should allow users to add run-specific instructions after a workflow has already started.

Example:

- "For this order, prioritize speed over cost."
- "If shipment is delayed, escalate immediately."
- "Do not contact the customer without human review."

These instructions should become part of the run context.

End-of-Run Output

When the workflow finishes, the system should generate:

- final summary,
- important actions taken,
- key learnings,
- feedback or recommendations.

This can be produced by the main agent as a final step.

Good-to-Have

These are not mandatory, but are strong additions:

- agent-generated wake-up guidance for the classifier,
- unknown-event escalation behavior,

- `continue_as_new` for very long histories,
- better memory compaction strategy,
- multiple supervisor templates,
- richer run analytics.

Suggested System Design

A clean implementation could have:

- FastAPI backend for workflow and run management,
- Temporal workflow per order,
- signal handlers for incoming events,
- one main agent runtime,
- one lightweight classifier or wake-up policy,
- persistence for runs, timeline, memory, and final output,
- Next.js UI for configuration and monitoring.

Suggested APIs

Your exact API design is up to you, but a clean version could include:

- `POST /api/supervisors`
- `GET /api/supervisors/{id}`
- `POST /api/runs`
- `GET /api/runs`
- `GET /api/runs/{run_id}`
- `POST /api/runs/{run_id}/events`
- `POST /api/runs/{run_id}/instructions`
- `POST /api/runs/{run_id}/interrupt`
- `POST /api/runs/{run_id}/resume`

- `POST /api/runs/{run_id}/terminate`

Persistence

Store at least:

- supervisor configs,
- run records,
- run status,
- event timeline,
- action and activity history,
- memory summary,
- final summary and learnings.

A single activity log table is acceptable for storing:

- incoming events,
- wake-up decisions,
- sleep decisions,
- agent actions,
- manual instructions,
- final outputs.

Acceptance Criteria

A submission is successful if:

- one Temporal workflow is started per order,
- order events can be sent into the workflow as signals,
- the agent can wake on start, signal, and scheduled wake-up,
- the agent can sleep and wake later,
- the agent can execute the required business actions,

- those actions are stored as activity records,
- the UI shows event history and action history for a run,
- the run stores timeline and compact memory,
- the UI can inject events and additional instructions,
- the workflow produces a final summary with learnings and feedback.

Scope Boundaries

You do not need to build:

- real commerce integrations,
- real messaging integrations,
- authentication,
- multi-tenant production hardening,
- advanced retrieval systems,
- multiple cooperating agents,
- polished design.

Keep the scope small and solid.

Deliverables

Please submit:

- source code,
- a README with setup instructions,
- a short architecture note,
- a walkthrough video showing the product working.

The walkthrough should show:

- creating a supervisor config,
- starting an order run,

- sending events into the workflow,
- the agent going to sleep and waking up,
- business action execution,
- adding extra instructions to a live run,
- interrupting or terminating a run,
- final summary, learnings, and feedback.

Evaluation Criteria

We will look at:

- architecture and clarity of design,
- Temporal usage,
- long-running workflow modeling,
- signal handling,
- quality of agent orchestration,
- memory and timeline design,
- frontend usability,
- code quality,
- documentation,
- overall product thinking.

Notes

A smaller implementation with a clean architecture and a reliable demo is better than a larger unfinished system.

Focus on building a believable POC, not a production platform.